

5. Perceptron Model Implementation: Build a single-layer perceptron using NumPy to understand the basic working of a neuron.

A perceptron is the simplest possible neural unit:

- It takes inputs
- Applies weights + bias
- Passes the result through a step activation
- Learns by adjusting weights using the perceptron learning rule

This is exactly the kind of foundational intuition that strengthens your work on **KrishiSuraksha**, where you're already using deeper models like MobileNetV2.

✓ Core Concepts

- Weighted sum: $z = w \cdot x + b$
- Activation function: Step function $\rightarrow$ outputs 0 or 1
- Learning rule:

$$w = w + \eta(y_{\text{true}} - y_{\text{pred}})x$$

```
def perceptron_train(X, y, lr=0.1, epochs=10):  
    n_features = X.shape[1]  
  
    weights = np.zeros(n_features)  
    bias = 0  
  
    for _ in range(epochs):  
        for i in range(len(X)):  
            linear_output = np.dot(X[i], weights) + bias  
            y_pred = step_function(linear_output)  
  
            # update  
            update = lr * (y[i] - y_pred)  
            weights += update * X[i]  
            bias += update  
    return weights, bias
```

Train on OR Logic Gate

```
# OR gate dataset
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 1])

# Train
p = Perceptron(input_dim=2, lr=0.1, epochs=10)
p.fit(X, y)

# Test
print("Predictions:", p.predict(X))
print("Weights:", p.weights)
print("Bias:", p.bias)
```

Prediction Function

```
def perceptron_predict(X, weights, bias):
    linear_output = np.dot(X, weights) + bias
    return step_function(linear_output)
```

Training the Perceptron

```
weights, bias = perceptron_train(X, y, lr=0.1, epochs=10)
print("Weights:", weights)
```

Testing

```
predictions = perceptron_predict(X, weights, bias)
print("Predictions:", predictions)
```

Output

Weights: [0.2 0.2]

Bias: -0.1

Predictions: [0 0 0 1]